Missing Values: A Comprehensive Guide

Detecting, diagnosing and handling missing data in machine learning

Written as a learning reference alongside the Home Credit loan default project.

Contents

1 Why missing data matters

2 Detecting missing data (including the hidden kind)

3 The three mechanisms: MCAR, MAR, MNAR

4 Diagnosing the mechanism in practice

5 Methods in depth: deletion, simple imputation, indicators, groups, time series, KNN, iterative, multiple imputation, matrix methods, deep learning, native handling

6 Which method for which model

7 Doing it correctly: leakage, pipelines, cross-validation, serving

8 Choosing between strategies: how to evaluate

9 Special situations

10 Common mistakes checklist

11 A decision guide

12 Worked example on this project

13 Glossary and further reading

1. Why missing data matters

Real datasets are almost never complete. Missing values cause three kinds of trouble:

- **Technical failure.** Many algorithms (linear and logistic regression, SVMs, most neural networks, scikit-learn's older tree implementations) raise an error or silently mishandle NaN.
- **Bias.** If the rows that are missing differ systematically from the rows that are present, then dropping or naively filling them gives a model that learns the wrong relationships. A model trained only on complete rows describes only the kind of person who fills in every field.
- **Lost efficiency and lost signal.** Dropping rows throws away information. And the fact that a value is missing is often itself informative: in the Home Credit data, applicants missing bureau information default about a third more often than others.

The goal is never just "get rid of the NaNs". It is to keep as much true information as possible without inventing information that is not there.

> **Rule of thumb.** Understand *why* a value is missing before choosing *how* to handle it. Most bad handling comes from skipping that step.

2. Detecting missing data

2.1 The obvious kind: NaN

pandas treats NaN, None, NaT and pd.NA as missing. isna() finds all of them.

```
df.isna().sum()                          # count per column
df.isna().mean().mul(100)                # percent per column
df.isna().sum(axis=1).value_counts()     # how many missing per ROW
```

2.2 The hidden kind: placeholders

Data providers often encode "unknown" as a real-looking value. isna() cannot see these, so they quietly become fake data. Typical placeholders:

- Text: "XNA", "Unknown", "N/A", "?", "-", empty or whitespace-only strings.
- Numbers: -1, 0, 99, 999, -999, 9999, or an impossible date such as 1900-01-01.
- Impossible values: negative ages, a person employed for about 1,000 years (365243 days in the Home Credit data).

How to hunt for them:

1. Read the data dictionary for each column's valid range and meaning.
2. For numeric columns, compare min, max and the 99th percentile; look for impossible values and for a max that is far from everything else.
3. Look for a single value repeated far more than the next most common one (value_counts().head()).
4. For text columns, scan value_counts(dropna=False) for suspicious labels, normalising case and whitespace first.
5. Check signs and units: a column that should be all negative or all positive but is not.

```
PLACEHOLDERS = {"xna", "unknown", "n/a", "na", "?", "", "none", "null"}

for col in df.select_dtypes("object"):
    cleaned = df[col].dropna().astype(str).str.strip().str.lower()
    n = cleaned.isin(PLACEHOLDERS).sum()
    if n:
        print(col, n)
```

Watch out. Call dropna() before astype(str). Otherwise real NaN becomes the string "nan" and gets counted as a placeholder.

2.3 Measuring and visualising

- **Per-column percentage** tells you how severe each column is. Bucket it (0%, under 5%, 5-20%, 20-50%, over 50%) to prioritise.
- **Per-row counts** show whether missingness is all-or-nothing or partial.
- **Missingness matrix** (missingno.matrix) shows horizontal bands when the same rows are missing the same fields.
- **Missingness correlation heatmap** (missingno.heatmap) shows which columns go missing together. Treat 100% as rounded; confirm with an exact comparison of the boolean masks.
- **Pattern table**: label each row with the names of its missing columns and group by that label. It shows which combinations actually occur.

3. The three mechanisms

Donald Rubin's classification (1976) describes how the chance of being missing relates to the data. It decides which methods are valid.

Mechanism	Meaning	Example	Consequence
MCAR Missing Completely At Random	Missingness is unrelated to any variable, observed or not	A lab sample is dropped on the floor	Complete rows are an unbiased random subset. Deletion loses power but does not bias estimates. Rare in practice.
MAR Missing At Random	Missingness depends only on *observed* variables	Younger applicants are less likely to report income, and age is recorded	Deletion can bias results. Methods that use the other columns (group, regression, KNN, MICE) can correct it.
MNAR Missing Not At Random	Missingness depends on the missing value itself or on unobserved factors	People with very high debt skip the debt question	No standard method fully corrects it. Use flags, domain knowledge, sensitivity analysis, and be honest about the limits.

A fourth case that matters in practice

Structurally missing (not applicable). The value does not exist. A car's age when the applicant has no car; an employer for a pensioner. This is neither random nor hidden: it is a fact about the applicant. Imputing a number here is wrong. The right treatment is a flag (or a category) and, for models that need a number, a neutral fill.

Important. Real data rarely fits one label for a whole dataset. Different columns, and even different parts of one column, can have different mechanisms. Decide per column or per block.

4. Diagnosing the mechanism in practice

You can never prove MAR versus MNAR from the data alone, because the evidence you would need is exactly the missing part. You can, however, gather evidence.

1. **Compare the target.** Group by the missing flag and compare the target rate, with group sizes. A real difference means missingness is informative (so it is not MCAR with respect to the target).
2. **Compare other features.** For a column with gaps, compare the distribution of other columns between missing and present rows (means, categories). Differences point to MAR.
3. **Model the missing flag.** Fit a small classifier that predicts "is this value missing?" from the other columns. If it predicts well (AUC clearly above 0.5), the mechanism is not MCAR.
4. **Formal test.** Little's MCAR test checks whether means differ across missingness patterns. It is available in R (naniar::mcar_test) and in some Python packages. A rejection means the data is not MCAR; it does not tell you MAR versus MNAR, and with large datasets it rejects almost always.
5. **Use domain knowledge.** Read the data dictionary and ask how the field is collected. This is often more decisive than any test.

A quick significance check for a rate difference

To judge whether two default rates differ by more than chance: the standard error of a rate p from n rows is sqrt(p(1-p)/n). For a gap between two groups, combine them: sqrt($se1^2$ + $se2^2$). Divide the gap by that value; above roughly 2 to 3, chance is an unlikely explanation. This is a rough guide that works for large groups and rates not near 0 or 1; for tiny groups say "too few rows".

```
se = lambda p, n: (p * (1 - p) / n) ** 0.5
gap = p_missing - p_present
z = gap / (se(p_missing, n_missing) ** 2 + se(p_present, n_present) ** 2) ** 0.5
```

5. Methods in depth

5.1 Deletion

- **Listwise (complete-case) deletion:** drop any row with a missing value. Valid under MCAR. Under MAR it can bias results, and with many columns it can delete nearly everything. It also cannot be used at prediction time: you must still score the incomplete customer.
- **Pairwise deletion:** use all available rows for each calculation separately (for example for each pair in a correlation matrix). Can produce inconsistent matrices.
- **Column deletion:** drop columns above a missingness threshold (often 50 to 80%). Only after checking that the column and its missingness carry no signal. A mostly empty column can still be very predictive through its flag.

Use when: a tiny fraction of rows is affected and the data is plausibly MCAR; or a column is nearly empty and uninformative.

5.2 Simple (single-value) imputation

Fill	Good for	Problems
Mean	Roughly symmetric numeric columns	Pulled by outliers; shrinks variance; weakens correlations
Median	Skewed numeric columns (income, amounts)	Still shrinks variance
Mode	Categorical columns	Over-represents the most common class
Constant (0, "Missing")	Cases where the constant has a meaning	Wrong if 0 is a real value; fake information otherwise

Single-value imputation distorts the distribution: every filled row gets the same number, so a spike appears at the mean or median, variance is understated and relationships with other variables are weakened. For prediction tasks this is often acceptable (especially with flags); for statistical inference it is not.

```
from sklearn.impute import SimpleImputer

imp = SimpleImputer(strategy="median", add_indicator=True)
X_train_imp = imp.fit_transform(X_train)   # learn medians on TRAIN only
X_test_imp = imp.transform(X_test)         # reuse them on test
```

5.3 Missing-indicator flags

Add a binary column that is 1 where the original value was missing. The model can then use the fact of missingness, even though the filled value itself is a guess.

- Essential for linear and logistic models when missingness is informative.
- Harmless but often redundant for tree models that already handle NaN.
- For block-structured missingness (many columns missing together) one flag per block is enough; 40 identical flags just add noise and size.
- Check for flags that are copies of an existing column (for example a missing-age flag that equals FLAG_OWN_CAR = N).

```
from sklearn.impute import MissingIndicator
flags = MissingIndicator(features="missing-only")   # one flag per column that had NaN
```

5.4 Categorical columns

- Add an explicit "Missing" category. Usually the best default: it preserves the information and needs no assumptions.
- Fill with the mode only if the gaps are tiny and random.
- For placeholders like XNA, either keep them as their own category or convert to NaN and then use the "Missing" category; decide based on whether the placeholder has its own meaning.
- Beware high-cardinality encodings (target encoding) with a missing category: the missing group can be small and noisy. Use smoothing.

5.5 Sentinel (out-of-range) values

Fill with an impossible value such as -999 so that a tree can isolate the missing rows. It works for trees, but for linear, distance-based and neural models the sentinel is treated as a real, extreme number and ruins the fit. Prefer a flag plus a normal fill.

5.6 Group-wise imputation

Fill each missing value with a statistic of a related subgroup, for example median income by occupation and education. This exploits MAR structure and keeps the fill more realistic than one global value. Compute the group statistics on training data only, and have a fallback (global median) for unseen or tiny groups.

```
medians = train.groupby("OCCUPATION_TYPE")["AMT_INCOME_TOTAL"].median()
fill = df["OCCUPATION_TYPE"].map(medians)
df["AMT_INCOME_TOTAL"] = df["AMT_INCOME_TOTAL"].fillna(fill)
df["AMT_INCOME_TOTAL"] = df["AMT_INCOME_TOTAL"].fillna(train["AMT_INCOME_TOTAL"].median())
```

5.7 Time series and ordered data

- **Forward fill** (carry the last value forward): safe for forecasting because it only uses the past.
- **Backward fill and interpolation** use future values. Fine for analysis of historical data, but they *leak the future* if used to build features for forecasting.
- **Linear, spline and time-weighted interpolation** suit smooth signals.
- **Seasonal fill** (same hour last week) suits seasonal data.
- Long gaps are better flagged than interpolated.

```
s.ffill()                          # past only: safe for forecasting
s.interpolate(method="time")       # uses past and future: analysis only
```

5.8 K-nearest neighbours (KNN) imputation

For each row with a gap, find the k most similar rows (using the observed columns) and average their values for the missing column. It can capture local structure and non-linear relationships.

- Needs scaled features; unscaled large-valued columns dominate the distance.
- Cost grows quickly with rows and columns: slow on hundreds of thousands of rows.
- Sensitive to the choice of k and to irrelevant columns.
- Categoricals need encoding first.

```
from sklearn.impute import KNNImputer
imp = KNNImputer(n_neighbors=5, weights="distance")
```

5.9 Iterative imputation (MICE-style)

Multivariate Imputation by Chained Equations. Start by filling every gap simply. Then, for each column in turn, train a model that predicts it from all the other columns (using the currently filled values), and replace its gaps with the predictions. Repeat for several rounds until values stabilise.

- Uses relationships between columns, so it is usually more accurate than a single fill when columns are strongly related.
- Slower and more complex; can fail to converge; sensitive to the model chosen for each column.
- Single predictions understate uncertainty (they are the expected value, with no noise). That is fine for prediction, not for inference.

```
from sklearn.experimental import enable_iterative_imputer   # noqa: F401
from sklearn.impute import IterativeImputer
imp = IterativeImputer(max_iter=10, random_state=42)
```

5.10 Multiple imputation

Instead of one filled dataset, create m (commonly 5 to 20) datasets, each with the gaps filled by random draws from a model of the missing values. Analyse each one separately and combine the results with **Rubin's rules**:

- The pooled estimate is the average of the m estimates.
- The total variance is the average within-imputation variance plus (1 + 1/m) times the between-imputation variance.

The between-imputation part captures the extra uncertainty caused by the missing data, which single imputation hides. It is the standard approach for *statistical inference* (confidence intervals, p-values). For pure prediction it is rarely worth the cost: single imputation plus flags, or native NaN handling, is typical. Related: **predictive mean matching (PMM)** fills a gap with an actual observed value from a similar row, which keeps imputed values realistic (for example, integers stay integers and values stay within the observed range).

5.11 Matrix and model-based methods

- **EM (expectation-maximisation)** estimates the parameters of an assumed distribution (often multivariate normal) in the presence of missing data, then fills values from it.
- **Low-rank matrix completion** (for example SoftImpute, SVD-based) assumes the data lies near a low-dimensional structure and reconstructs the gaps. Popular in recommender systems.
- **Deep learning** (denoising autoencoders, GAIN, VAEs) can model complex dependencies on large data, but needs more data, tuning and care, and rarely beats simpler methods on tabular problems.

5.12 Letting the model handle missing values

Model	How it handles NaN
XGBoost	At each split learns a default direction for missing values
LightGBM	Treats missing as its own group at splits (use_missing=True by default)
CatBoost	Handles missing with built-in strategies (for example Min / Max modes)
scikit-learn HistGradientBoosting	Native NaN support; learns which side missing values go
scikit-learn trees / random forests	Newer versions support NaN natively (check your installed version's documentation)
Linear / logistic regression, SVM, KNN, most neural nets	No native support: impute first

Native handling preserves the signal in the gap without any imputation choices. The trade-off is less control and, for some libraries, less transparency about what they do. Always confirm in the current documentation for your installed version.

6. Which method for which model

Model family	Recommended approach
Linear / logistic regression	Median or group fill (+ log transform for skew) plus missing flags; scale afterwards
Regularised linear (Lasso, Ridge)	Same as above; flags are cheap and useful
KNN, SVM	Impute, then scale; flags if missingness is informative
Neural networks	Impute and scale; flags; embedding for categories with a "Missing" level
Random forest	Native NaN if available; otherwise median fill plus flags
Gradient boosting (XGBoost, LightGBM, CatBoost, HistGB)	Leave NaN as is; add a few block-level flags if useful; convert placeholders to NaN
Statistical inference / reporting	Multiple imputation with Rubin's rules

7. Doing it correctly

7.1 Data leakage

Imputation learns parameters from data: a median, a group mean, a KNN neighbourhood, a regression. If those are learned from rows that later end up in the test set, test information has leaked into training and your scores are optimistic.

1. Split into train and test first (stratify if the target is imbalanced).
2. Fit the imputer on the training data only.
3. Apply (transform) the fitted imputer to validation and test data.
4. During cross-validation, refit the imputer inside each fold. Putting it in a scikit-learn Pipeline does this automatically.

> **Generalises.** The same rule applies to everything that learns from data: scalers, encoders, feature selection, outlier thresholds.

7.2 A complete pipeline

```
import numpy as np
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

num_cols = X_train.select_dtypes("number").columns
cat_cols = X_train.select_dtypes("object").columns

numeric = Pipeline([
    ("impute", SimpleImputer(strategy="median", add_indicator=True)),
    ("scale", StandardScaler()),
])
categorical = Pipeline([
    ("impute", SimpleImputer(strategy="constant", fill_value="Missing")),
    ("encode", OneHotEncoder(handle_unknown="ignore")),
])

prep = ColumnTransformer([
    ("num", numeric, num_cols),
    ("cat", categorical, cat_cols),
])

model = Pipeline([("prep", prep), ("clf", LogisticRegression(max_iter=1000))])
scores = cross_val_score(model, X_train, y_train, cv=5, scoring="average_precision")
```

Convert placeholders to NaN *before* this pipeline, in a deterministic step that is also applied to new data (for example a FunctionTransformer at the start of the pipeline).

7.3 Serving and new data

- The exact same preparation must run at prediction time. Save the whole fitted pipeline, not just the model.
- A column that was never missing in training may be missing in production. Decide the behaviour (for example make the imputer fit all columns, or set a safe default) and test it with a row containing NaN everywhere.
- Monitor the missing rate per column over time. A sudden jump (for example a broken upstream feed) can silently wreck predictions.
- A new category unseen in training must not crash the encoder (handle_unknown).

8. Choosing between strategies: how to evaluate

There is rarely a universally best method. Compare them empirically.

1. **Downstream metric (the one that matters).** Build a pipeline per strategy (median + flag, native NaN, iterative, and so on) and compare cross-validated scores on the metric you care about. For an imbalanced target such as default, use PR-AUC or ROC-AUC, not accuracy.
2. **Mask-and-recover experiment.** Take rows that are fully observed, hide some known values at random, impute, and measure the error against the truth (RMSE for numbers, accuracy for categories). Useful for comparing imputers directly, but it assumes the hidden values are MCAR, which may not match reality.
3. **Distribution checks.** Compare the histogram, mean, variance and correlations of a column before and after imputation. A large spike at one value means the fill is distorting the data.
4. **Sensitivity analysis.** Re-run with different plausible assumptions (for example fill a MNAR column optimistically and pessimistically). If conclusions change, report that honestly.

> **Rule of thumb.** Keep the validation scheme identical across strategies and fix random seeds, so that differences come from the missing-value handling and not from the split.

9. Special situations

- **Very high missingness (over 50 to 70%).** Imputing is mostly guessing. Prefer a flag plus native NaN, or a coarse binned version of the column with a "Missing" bin.
- **Missing target.** For supervised learning, rows with a missing label cannot be used for supervised training. Do not impute the target (semi-supervised learning is a separate topic).
- **Skewed numbers.** Median, or log-transform before a regression-based imputer, then transform back.
- **Discrete or bounded variables.** Mean imputation creates impossible values (2.7 children). Use median/mode, PMM, or round and clip.
- **Constraints between columns.** If A must be at most B, or parts must add up to a total, impute in a way that respects it, or fix it afterwards.
- **Imbalanced targets.** Check the missing-versus-present target rate, as the signal may sit entirely in the gap. Do not let imputation remove it.
- **Outliers.** Means and regression imputers are sensitive to them. Handle extreme values (or use medians) first.
- **Correlated duplicates.** Columns like _AVG, _MODE and _MEDI that are missing on the same rows carry one piece of information. Consider keeping one, or summarising the block.
- **Panel or grouped data.** Rows from the same person or entity should stay together in the train/test split, otherwise imputed information leaks across.
- **Ethical and fairness angle.** Missingness often correlates with group membership (for example income level, age). Handling it carelessly can create or hide disparities. Check performance by subgroup.

10. Common mistakes checklist

Mistake	Why it hurts	Fix
Imputing before train/test split	Test information leaks into training	Fit imputers inside a Pipeline after splitting
Not converting placeholders to NaN	Fake values are treated as data	Hunt placeholders; convert first
Mean-filling skewed columns	Outliers distort the fill	Use median or transform

Mistake	Why it hurts	Fix
Dropping rows by default	Bias and large data loss	Check how many and whether MCAR; otherwise impute
Dropping informative columns	Loses the signal in the gap	Test missing vs present target rate first
Imputing "not applicable" values	Invents facts (a car for a non-owner)	Flag or category; neutral fill if required
One flag per column in a block	Redundant, noisy features	One flag per block
Sentinel values with linear models	Treated as a huge real number	Flag plus a normal fill
Reporting inference with single imputation	Understated uncertainty	Use multiple imputation
Ignoring missingness at serving	Silent failures in production	Test with all-NaN rows; monitor missing rates
Comparing methods on different splits	Differences are noise	Same folds and seeds for every strategy
Reading rates without counts	Tiny groups give extreme rates	Always show n beside every rate

11. A decision guide

1. **Is it really missing?** Hunt for placeholders and impossible values; convert them to NaN.
2. **Does it make sense for the value to exist?** If not (not applicable), use a flag or category; do not invent a number.
3. **How much is missing?** A tiny fraction: simple handling is fine. Large: prefer flags and native NaN over elaborate imputation.
4. **Is missingness informative?** Compare the target rate with counts. If yes, keep a flag and make sure the method does not erase the signal.
5. **Do columns go missing together?** If so, treat them as a block: one flag, one strategy.
6. **Which model?** Tree boosting: leave NaN. Linear, SVM, KNN, neural: impute (and scale).
7. **Prediction or inference?** Prediction: single imputation plus flags is usually enough. Inference: multiple imputation.
8. **Put it in a Pipeline**, fit on training data only, and evaluate with cross-validation.
9. **Compare two or three sensible strategies** on the downstream metric, and pick the simplest one that performs well.
10. **Document** what you decided and why. This is what you will explain in an interview.

12. Worked example on this project

Findings from the Home Credit application_train data (307,511 rows), and how each could be handled. These are options to weigh, not final answers.

Block	Finding	Treatment options
Housing (47 cols, _AVG / _MODE / _MEDI and others)	Strongly correlated missingness, not one clean block (a wide spread of per-row counts). Missing is linked to higher default (8.6% vs 6.9% for COMMONAREA_AVG).	Native-NaN trees with a block flag; or median fill plus a flag for linear models; consider keeping one of the _AVG / _MODE / _MEDI variants.
Credit bureau (6 cols)	Identical missingness across columns. Missing linked to higher default (10.3% vs 7.7%).	One flag; zero or median fill depending on what "no record" means.
Social circle (4 cols)	Identical missingness; only 1,021 rows; lower default when missing (3.5% vs 8.1%).	A flag or just a fill; small impact because the group is tiny.
EXT_SOURCE_1/2/3	Independent missingness; strong predictors; EXT_SOURCE_1 missing: 8.5% vs 7.5%.	Treat each separately: a flag plus median fill, or native NaN.
OWN_CAR_AGE	Largest single missing pattern (about 130k rows alone, 9.6% default). Likely means no car; verify against FLAG_OWN_CAR.	Not applicable: flag from FLAG_OWN_CAR plus neutral fill or native NaN.
DAYS_EMPLOYED and ORGANIZATION_TYPE	365243 and XNA on exactly the same 55,374 rows (18%); 99.96% are pensioners; default 5.4% vs 8.7%.	Convert 365243 to NaN in the pipeline; rely on income type and/or a flag; keep XNA as its own category or use the flag.
CODE_GENDER = XNA (4 rows), NAME_FAMILY_STATUS = Unknown (2 rows)	Negligible size.	Convert to NaN and let the standard handling cover it.

Suggested plan

1. Split train/test (stratified on TARGET) before any fitting.
2. A cleaning step: convert 365243 and XNA/Unknown placeholders to NaN.

3. Baseline A: logistic regression with the pipeline from section 7.2.
4. Baseline B: gradient boosting with native NaN plus a few block flags.
5. Compare with cross-validated PR-AUC and ROC-AUC; keep the simpler one if they are close.
6. Write down the chosen treatment per block and the evidence for it.

13. Glossary and further reading

Term	Meaning
Imputation	Filling a missing value with an estimate
Listwise deletion	Dropping any row with a missing value
Missing indicator	A 0/1 column marking where a value was missing
MCAR / MAR / MNAR	Missingness unrelated to the data / depends on observed data / depends on unobserved data
Structural missing	The value does not exist for that row (not applicable)
Sentinel / placeholder	A fake value standing in for "unknown"
MICE	Multivariate Imputation by Chained Equations
Multiple imputation	Several imputed datasets, analysed and pooled
Rubin's rules	Formulas to pool estimates and variances across imputations
PMM	Predictive mean matching: fill with an observed value from a similar row
Data leakage	Information from outside the training data influencing the model

Further reading

- Little and Rubin, *Statistical Analysis with Missing Data* (the classic text).
- Stef van Buuren, *Flexible Imputation of Missing Data* (practical, covers MICE, available free online).
- scikit-learn user guide, section "Imputation of missing values", and the documentation of SimpleImputer, KNNImputer, IterativeImputer and HistGradientBoostingClassifier.
- Documentation of XGBoost, LightGBM and CatBoost on how they treat missing values.
- The missingno library documentation for visualising missingness.